CS 330 — SOFTWARE ENGINEERING · CLASS 4

Agile Development & Software Process Improvement

Tuesday, October 13, 2026

Topic 2 — Software Processes & Standards (part 2)

LO1 (Learning Outcome 1) — debate the goals, benefits and drawbacks of the industry standards for software process and process improvement

1 Agility

2 Scrum

3 Extreme Programming and Kanban

4 Planning and metrics

5 Process improvement

Educational material — Cyber.SoHo Educational Hub · © 2026 Cyber.SoHo

Use is governed by the Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy. All teams, products and numbers are fictional.

The hour at a glance

0:00–0:10	Lesson 1 — Agility
0:10–0:14	🎯 Fun-quiz
0:14–0:22	Lesson 2 — Scrum
0:22–0:26	👁 Visualize-it
0:26–0:34	Lesson 3 — **XP** (Extreme Programming) and Kanban
0:34–0:44	Lesson 4 — Planning and metrics
0:44–0:48	✋ Try-it
0:48–0:56	Lesson 5 — Process improvement
0:56–1:00	🧘 Zen-break + 🃏 flash-cards

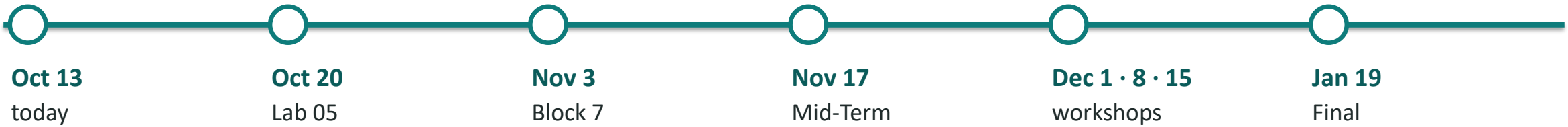
Today's question

- What does a team commit to when it says it is agile?
- How does Scrum organise that commitment?
- Which practices make short cycles possible — and how are they planned with numbers?
- How does an organisation improve its process?

No new Block today: the Studio is coached time for Block 3.

Where today's ideas come back

- Today: coached Studio — finish Block 3 (optional: five user stories)
- Oct 20 · Lab 05 / Block 5: 6–8 user stories, then the requirements
- Nov 3 · Block 7: Definition of Done → exit criteria; tests first; pairs
- Nov 17 · Mid-Term: velocity computation; user stories with acceptance criteria
- Dec 1 · 8 · 15 · workshops: written stand-up, one Block in progress, reviews
- Jan 19 · Final: the **LO1** (Learning Outcome 1) debate on standards and improvement



1

LESSON 1 · 0:00–0:10 · 10 MINUTES · SLIDES 4–12

Agility

the Agile Manifesto (four values, twelve principles), agile versus plan-driven development, and when agile fits

Goal of the lesson

Define agility; state the four values of the Agile Manifesto and group its twelve principles; compare agile and plan-driven development; use the five factors of Boehm and Turner to decide how agile a project can be.

What agility means

- **Agility**: respond to change quickly and at low cost, while still delivering working software
- Three conditions: talk with every stakeholder · bring the customer into the team · the team controls its own work
- **Agile software development**: short iterations, a working increment often, feedback decides the next step
- Why: users discover what they need when they see a first version

Plan once, then build

A change discovered late means rework of finished documents, design and code.

Build in short cycles

A change discovered late is the input of the next iteration.

The Agile Manifesto (2001)

- February 2001, Snowbird, Utah: seventeen practitioners
- They came from Scrum, **XP** (Extreme Programming), Crystal, Feature-Driven Development and other lightweight methods
- They agreed on four values and twelve principles
- A statement of priorities, not a method: no iteration length, no roles

2001
Snowbird, Utah

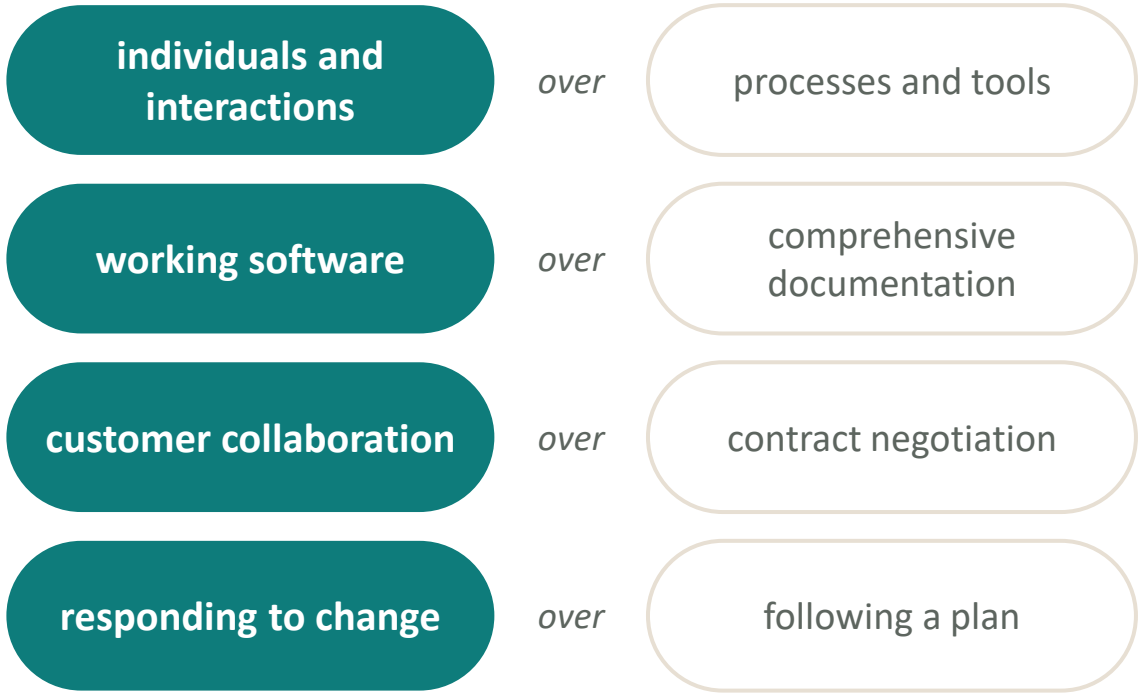
17
signatories

4
values

12
principles

The four values

- The items on the left are valued **more**
- The items on the right **still have value**
- “We are agile, so we write no documentation” is a misreading
- Agile documentation: the documents the team needs, when it needs them, short enough to stay correct



The twelve principles, in four groups

- Customer and value (1–3): early, continuous delivery · welcome change · frequent delivery, the shorter timescale preferred
- People (4–6): business and developers together daily · motivated, trusted people · face-to-face conversation
- Progress and pace (7–8): working software measures progress · sustainable development, a constant pace
- Quality and reflection (9–12): technical excellence · simplicity · self-organising teams · regular reflection

Customer and value
principles 1–3

People
principles 4–6

Progress and pace
principles 7–8

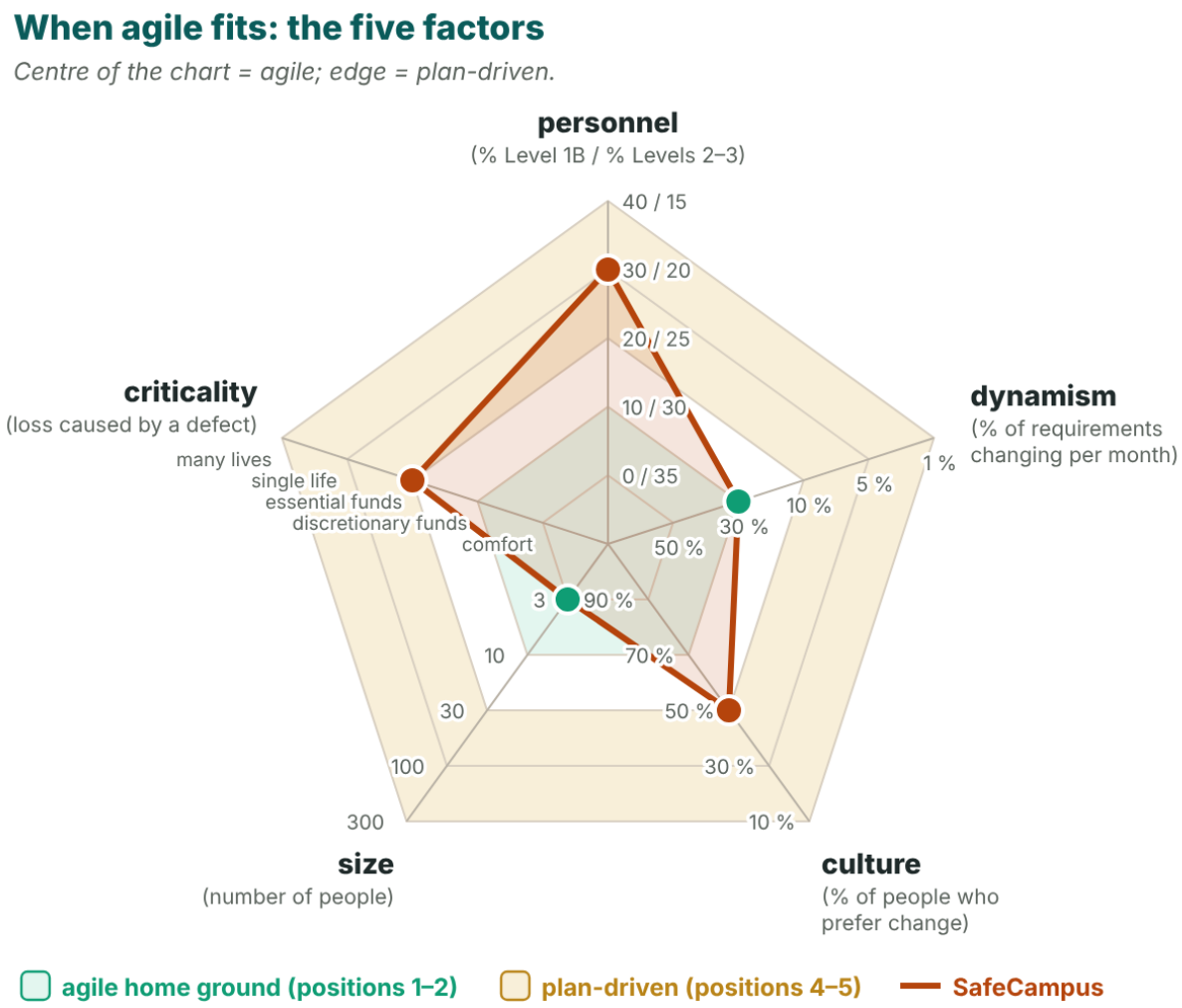
Quality and reflection
principles 9–12

Agile versus plan-driven

	Agile	Plan-driven
Requirements	emerge and change during the project	fixed and approved early
Planning	re-planned at every iteration	detailed plan at the start
Delivery	a working increment every few weeks	one release, or a few, at the end
Documentation	just enough, written when needed	comprehensive, written before the code
Customer	works with the team continuously	reviews at milestones
Progress measured by	working software	completed phases and documents
A change is	expected input	handled by a formal change request

When agile fits (1/2): five factors

- Boehm and Turner (2003): describe a project by five factors
- **Size** — how many people · **Criticality** — worst loss a defect could cause
- **Dynamism** — share of the requirements that change each month
- **Personnel** — who can adapt a method, who needs a procedure
- **Culture** — preference for frequent change or for fixed rules
- Centre of their chart: agile home ground · edge: plan-driven home ground



When agile fits (2/2): most projects are hybrids

- Most projects sit between the two home grounds
- Hybrid: agile where the factors allow it, written discipline where a factor pulls outward
- SafeCampus: personnel 4, criticality 3, culture 3 → reviews, a Definition of Done, extra checks on incident data
- Government of Canada Digital Standards: “Iterate and improve frequently” — with the controls the law requires

Agile where it fits

short iterations, the client sees every increment

Discipline where it pulls out

written reviews and checks for the outward factors

Public sector, Canada

agile methods required by a federal standard, with audits and approvals kept

Lesson 1 after class: watch, try, run

▶ Watch — 15 minutes or shorter

★ **What is Agile?** 11:56 ▶ watch

Mark Shead

Agile vs (versus). Waterfall Methodology Explained 10:46 ▶ watch

TeamGantt

What is the Agile Manifesto? 12:58 ▶ watch

Grow with Google

Real-life example

SafeCampus: how agile can Team Heron be?

Comes back in:

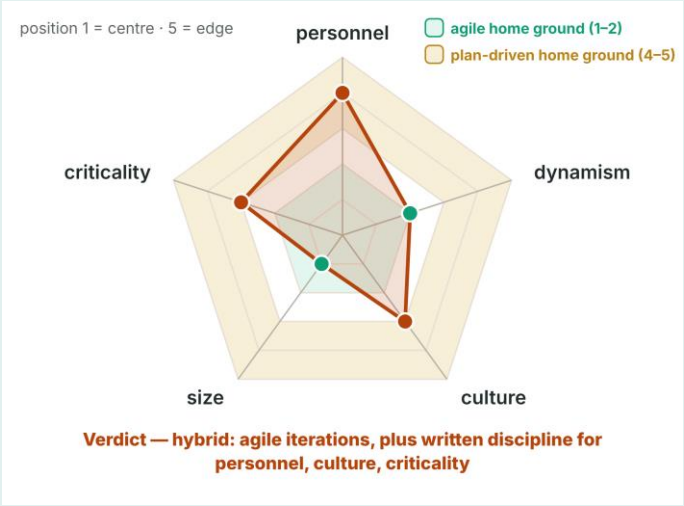
- Block 3
- Lab 05 / Block 5
- Mid-Term Part C and the Final's LO1 (Learning Outcome 1) debate question

Team Heron and SafeCampus are fictional.

Demo program

demo1_safecampus_agility_profile.py

runs step by step in the module



5 flash-cards · ✎ 6 formative questions (4 multiple-choice, 2 multiple-select) — in the module and the Study Hub. **Formative: nothing is graded.**

Active break — Fun-quiz: Agile or not? Three statements



4:00

0:10–0:14

Read each statement on the screen; vote with a thumb up (agile), a thumb down (not agile), or a flat hand (it depends).

1. “We are agile, so we do not write documentation.”
2. “Every two weeks we show the working product to the cafeteria manager and change the plan with what we learn.”
3. “All the requirements are signed in September; any change needs a committee's approval.”

2

LESSON 2 · 0:14–0:22 · 8 MINUTES · SLIDES 14–21

Scrum

accountabilities, events, artifacts and the Definition of Done

Goal of the lesson

Name the three accountabilities, the five events with their timeboxes and the three artifacts with their commitments; explain why only Done work counts; follow one Sprint from planning to retrospective.

Scrum: a framework built on empiricism

- **Scrum**: one small team, fixed-length Sprints, inspect and adapt every day and at the end of each Sprint
- Defined by the Scrum Guide (Schwaber and Sutherland), official edition of November 2020
- 2025 Expansion Pack: a companion, not a replacement
- **Empiricism**: decide on what is observed
- Three pillars: **transparency · inspection · adaptation**
- Deliberately short: 3 accountabilities · 5 events · 3 artifacts

3
accountabilities

5
events

3
artifacts

3
pillars

The Scrum Team: three accountabilities

- One small team, typically ten people or fewer, no sub-teams, no hierarchy
- **Product Owner** — maximises the value of the product; orders the Product Backlog; one person
- **Scrum Master** — the team's effectiveness; coaches; causes impediments to be removed; keeps the events in their timebox
- **Developers** — create a usable Increment every Sprint; plan the Sprint; follow the Definition of Done
- Your team of four: one Product Owner, one Scrum Master, four Developers

Product Owner

value of the product · Product Backlog

Scrum Master

effectiveness · impediments · events

Developers

the Increment · the plan · quality

The five events and their timeboxes

Event	Purpose	Maximum for a one-month Sprint
Sprint	container of the other events: produce a usable Increment	one month
Sprint Planning	why, what and how of this Sprint; write the Sprint Goal	8 hours
Daily Scrum	inspect progress toward the Sprint Goal; adapt the plan	15 minutes
Sprint Review	inspect the outcome with stakeholders; adapt the Product Backlog	4 hours
Sprint Retrospective	inspect the way of working; plan improvements	3 hours

Three artifacts, three commitments

- **Product Backlog** → commitment: **Product Goal**
- **Sprint Backlog** → commitment: **Sprint Goal**
- **Increment** → commitment: **Definition of Done**
- **Refinement**: adding detail to backlog items — a continuous activity, not an event
- Work counts only when it is Done



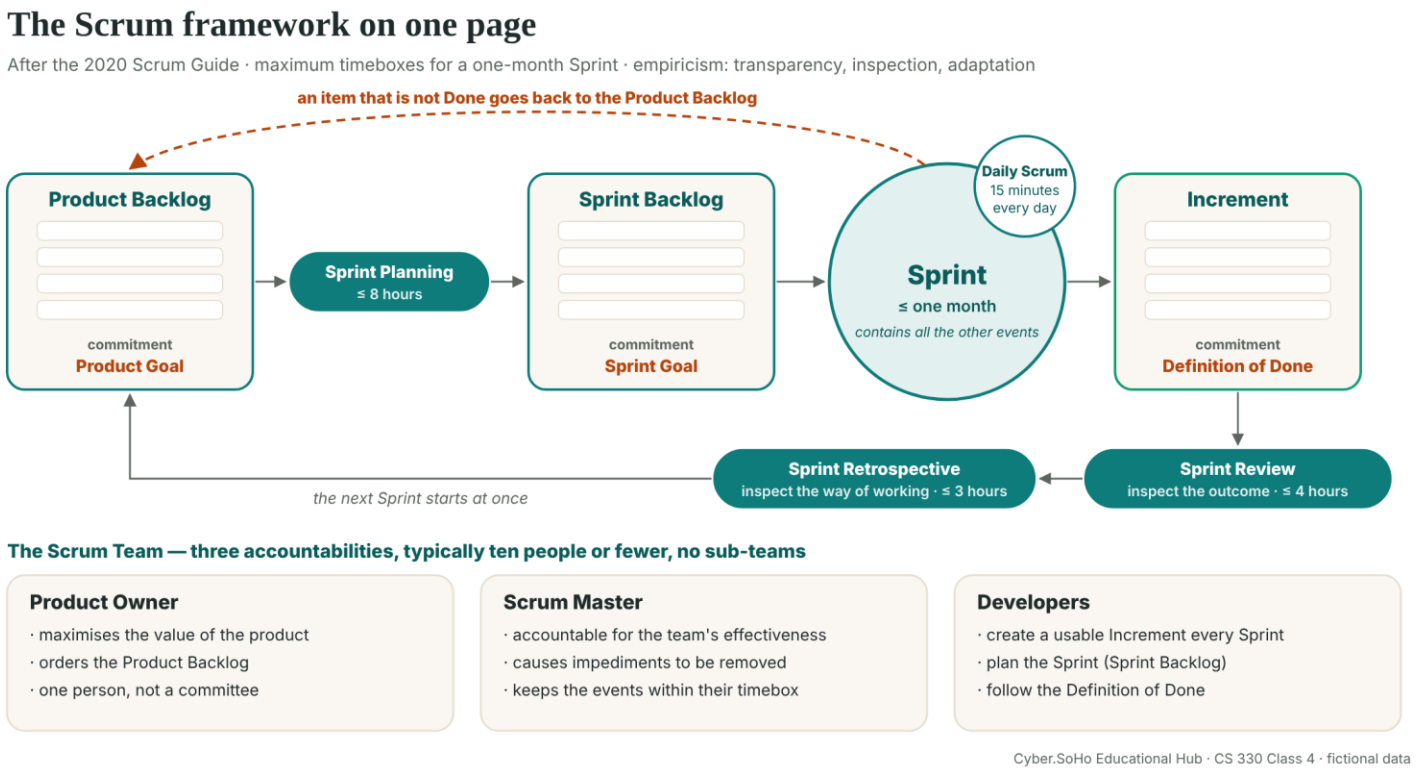
The Definition of Done

- A formal list of quality checks, written in advance, shared by the whole team
- An item that fails it cannot be released, or even shown at the Sprint Review → back to the Product Backlog
- Example: reviewed by a teammate · unit tests passing · acceptance criteria met · merged into main
- “Almost done” does not exist
- Block 7 (Nov 3): your Definition of Done becomes the exit criteria of the test plan



The Scrum loop — and this course

- Product Backlog → Sprint Planning → Sprint (a Daily Scrum each day) → Increment → Review → Retrospective → next Sprint
- Your Sprint 1: Oct 20 → Nov 8 · Sprint 2: Nov 24 → Dec 20
- December workshops open with a written stand-up: a written form of the Daily Scrum
- The exit ticket: a one-question retrospective



Lesson 2 after class: watch, try, run

▶ Watch — 15 minutes or shorter

★ **An Introduction to Scrum** 4:10 ▶ watch

Scrum.org

Scrum Essentials in Under 10 Minutes 10:16 ▶ watch

Scrum Alliance

How Scrum Works 4:33 ▶ watch

Scrum **Inc.** (Incorporated)

Real-life example

QueueLess, Sprint 1: why only Done work counts

Comes back in:

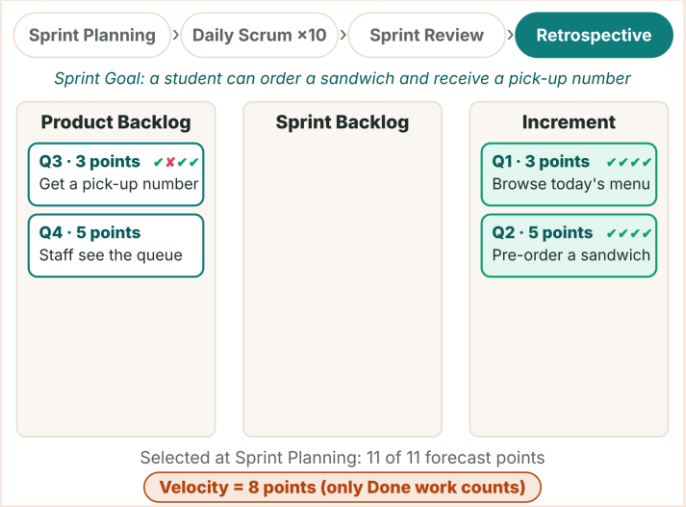
- Block 7
- Lab 05
- Lesson 4's forecast; Mid-Term Part B, the velocity computation
- The December finishing workshops open with it

Team Falcon and QueueLess are fictional.

Demo program

demo2_queueless_sprint.py

runs step by step in the module



5 flash-cards · ✎ 6 formative questions (4 multiple-choice, 2 multiple-select) — in the module and the Study Hub. **Formative: nothing is graded.**

Active break — Visualize-it: Draw the Scrum loop — with your names



4:00

0:22–0:26

1. One and a half minutes, alone, on paper: draw the Scrum loop from memory — the three artifacts, the five events and the arrows between them.
2. Thirty seconds: next to each accountability, write a teammate's name for Sprint 1 (October 20 → November 8): one Product Owner, one Scrum Master, every member a Developer.
3. Two minutes with a neighbour from another team: find one difference between your drawings and explain it in one sentence.

LESSON 3 · 0:26–0:34 · 8 MINUTES · SLIDES 23–30

3

Extreme Programming and Kanban

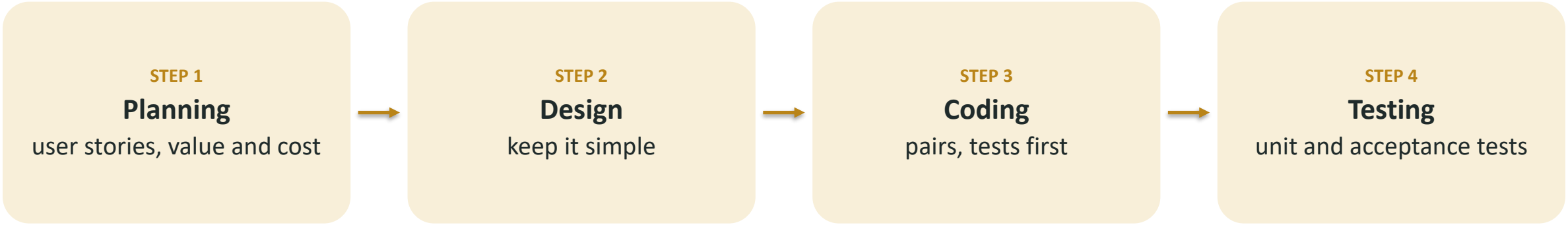
user stories, pair programming, **TDD** (Test-Driven Development), refactoring, continuous integration, small releases; Kanban flow and **WIP** (work in progress) limits

Goal of the lesson

Describe the values and the core practices of **XP** (Extreme Programming); explain the red–green–refactor cycle of **TDD** (Test-Driven Development); describe a Kanban board, a **WIP** (work in progress) limit and a pull system; use Little's Law to relate work in progress, throughput and cycle time.

Extreme Programming: values and process

- Kent Beck, late 1990s; *Extreme Programming Explained* (1999; 2nd edition with Cynthia Andres, 2004)
- Known good practices, applied all the time
- Five values: **communication · simplicity · feedback · courage · respect**
- Process: planning with user stories → simple design → coding in pairs, tests first → testing (unit + acceptance)



XP (Extreme Programming) practices (1/2): stories, small releases, pairs

- **User story**: As a ⟨user⟩, I want ⟨goal⟩, so that ⟨benefit⟩
- Card · conversation · confirmation (Ron Jeffries)
- **Small release**: working, tested software in users' hands often
- **Pair programming**: driver writes, navigator reviews each line; swap often
- Studies since about 2000: fewer defects, somewhat more total effort
- Lab 07 (Nov 3): first stories in pairs, keyboard swapped every 10 minutes

User story

As a student, I want to send a study request, so that we can prepare the Mid-Term together.

Small release

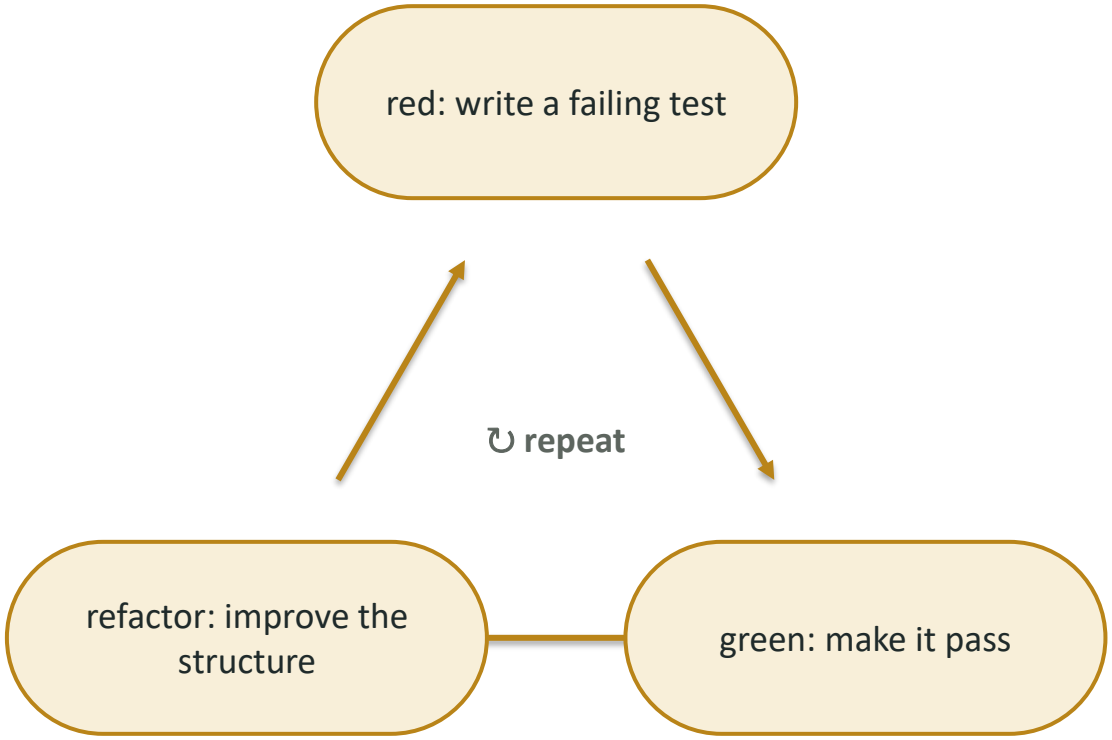
feedback arrives while the plan can still change

Pair programming

one keyboard, two people, roles swapped often

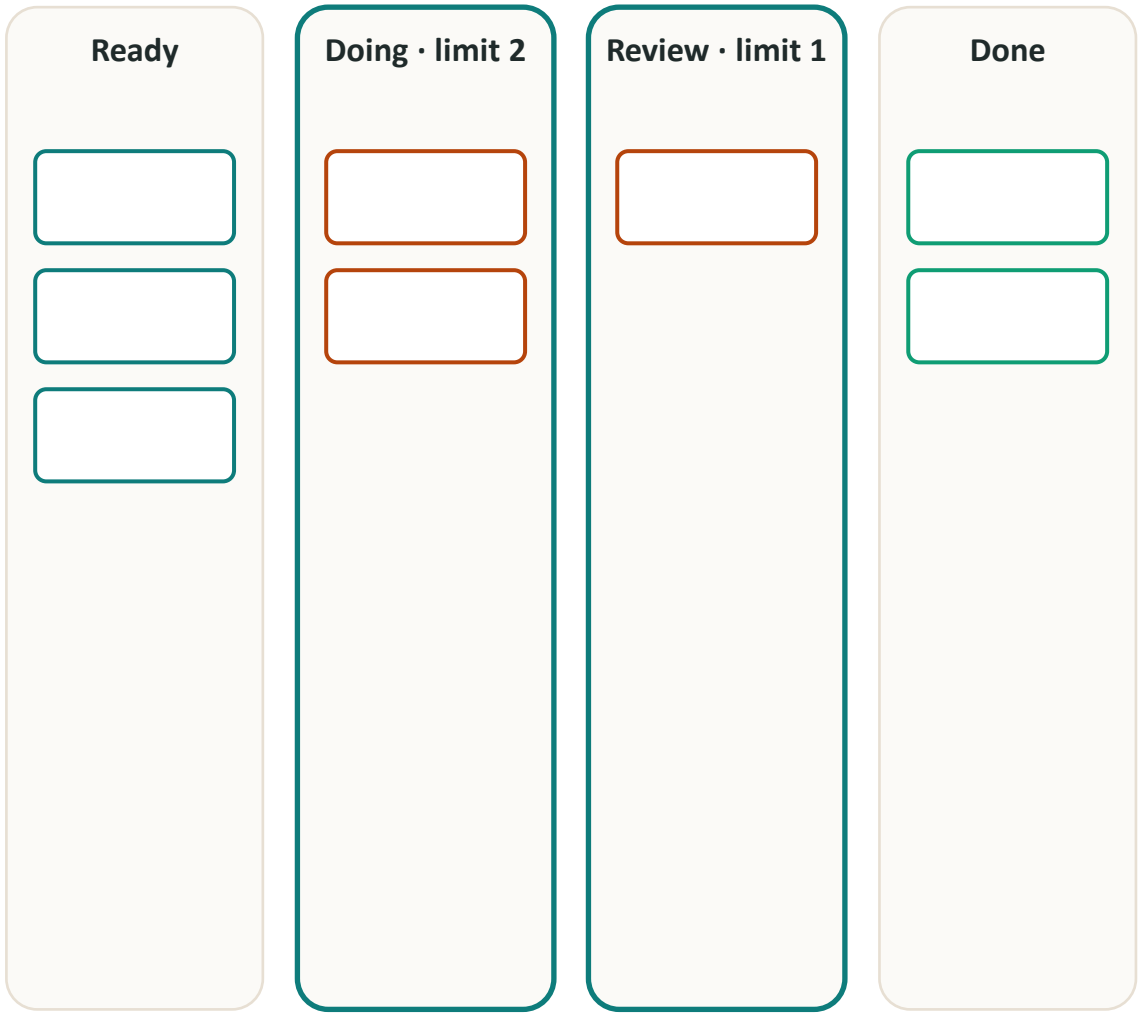
XP (Extreme Programming) practices (2/2): test first, refactor, integrate

- **TDD (Test-Driven Development)**: red (a failing test) → green (smallest code that passes) → refactor (better structure, same behaviour)
- **Refactoring**: change the internal structure, not the external behaviour
- **Continuous integration**: merge small changes to main at least daily; every merge builds and tests
- They depend on each other: tests make refactoring safe; integration runs all the tests
- Lab 07: one failing test first · Nov 24: a pipeline that tests every push



Kanban: visualise the flow, limit the work in progress

- From Toyota's production system; adapted to software by David J. Anderson in the 2000s
- Two current guides: *The Kanban Guide* (Coleman and Vacanti, May 2025) · Kanban University's *Official Guide to The Kanban Method*
- **Kanban board**: one column per step, one card per item
- **WIP (work in progress) limit**: maximum number of items in a column, or per person
- **Pull system**: start an item only when there is room under the limit
- No Sprints, no prescribed roles: improve one rule at a time



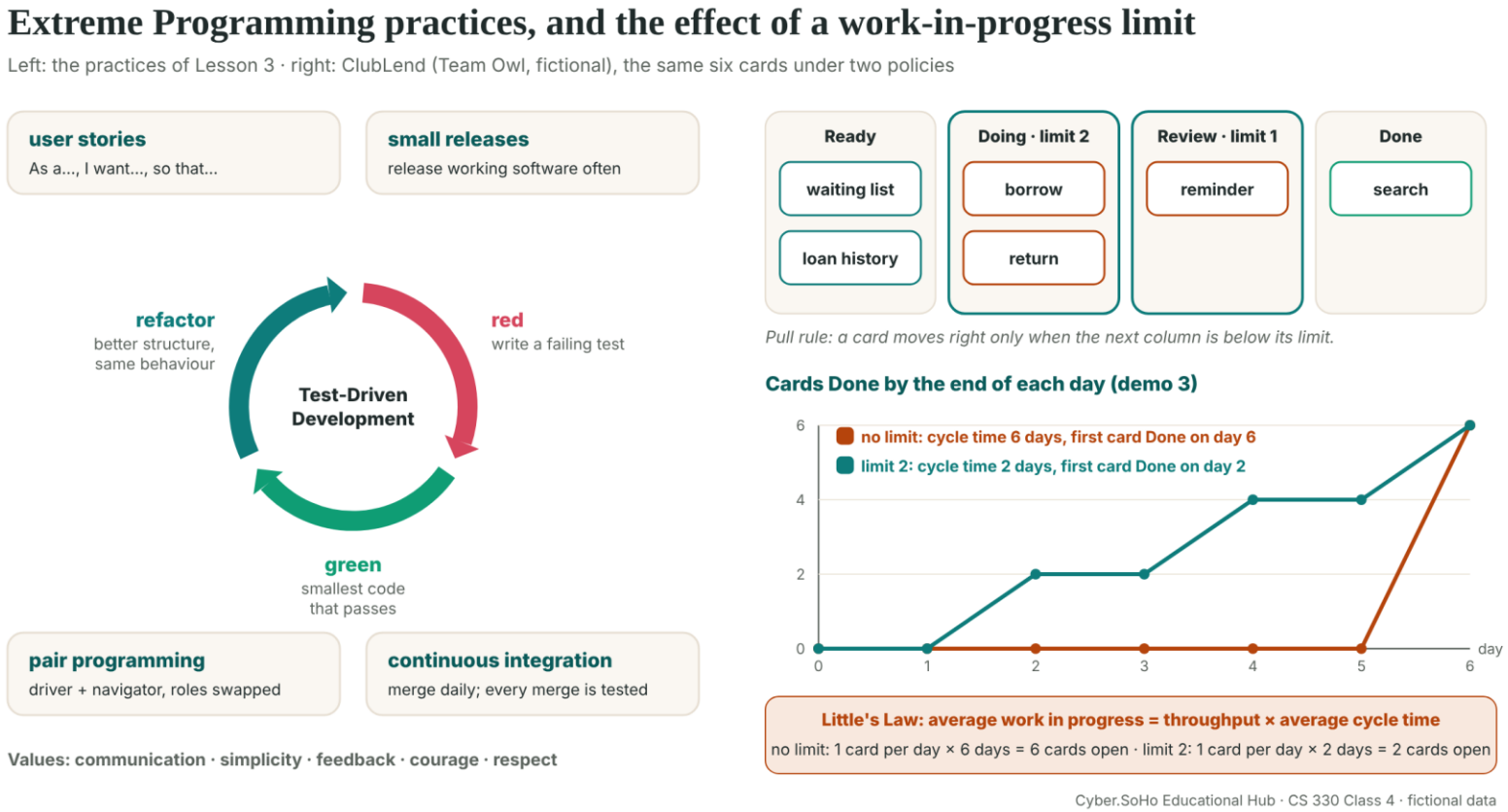
Measuring the flow: Little's Law

- **Throughput**: items finished per unit of time
- **Cycle time**: time from the start of an item to its finish
- Also tracked: work in progress, work item age
- **Little's Law** (1961): average work in progress = throughput × average cycle time
- 6 cards open, 1 finished per day → 6 days per card · 2 cards open → 2 days per card, same throughput

average work in progress = throughput × average cycle time

6 open ÷ 1 per day = 6 days · 2 open ÷ 1 per day = 2 days

Figure 3 — XP (Extreme Programming) practices and the effect of a WIP (work in progress) limit



Left: the **XP** (Extreme Programming) practices of the lesson around the red–green–refactor cycle. Right: the ClubLend board with a limit on Doing, and the cards Done per day under the two policies of the demo; Little's Law links the numbers.

Lesson 3 after class: watch, try, run

▶ Watch — 15 minutes or shorter

★ What is Extreme Programming (XP)? 10:18 ▶ watch

Online **PM** (Project Management) Courses - Mike Clayton

★ What is Kanban? 5:45 ▶ watch

Businessmap

Scrum vs (versus) Kanban - What's the Difference? 5:08 ▶ watch

Development That Pays

Real-life example

ClubLend: the same six cards, with and without a limit

Comes back in:

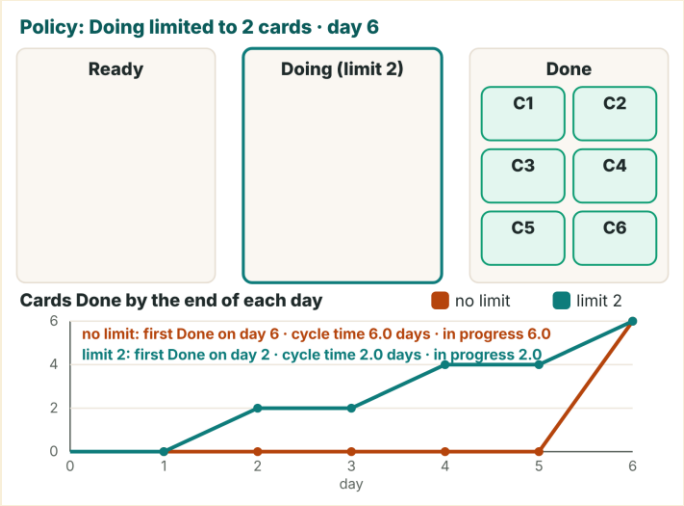
- The December finishing workshops: one Block in progress at a time, named in each written stand-up
- Lab 07: pairs implement one story at a time, test first
- Lab 07
- The Final's delivery and flow questions

Team Owl and ClubLend are fictional.

Demo program

demo3_clublend_kanban_flow.py

runs step by step in the module



5 flash-cards · ✎ 6 formative questions (4 multiple-choice, 2 multiple-select) — in the module and the Study Hub. **Formative: nothing is graded.**

4

LESSON 4 · 0:34–0:44 · 10 MINUTES · SLIDES 31–37

Planning and metrics

INVEST (Independent, Negotiable, Valuable, Estimable, Small, Testable) user stories, acceptance criteria, story points and planning poker, velocity, burndown charts — and the Numbers corner

Goal of the lesson

Check a user story with **INVEST** (Independent, Negotiable, Valuable, Estimable, Small, Testable); write acceptance criteria in the Given/When/Then form; explain story points and planning poker; compute the number of Sprints from the velocity; read a burndown chart.

Good user stories: INVEST (Independent, Negotiable, Valuable, Estimable, Small, Testable)

- Bill Wake, 2003: six checks for a story
- **I**ndependent · **N**egotiable · **V**aluable · **E**stimable · **S**mall · **T**estable
- Weak: “The system shall have a database of study sessions.” — a task, not a story
- Strong: “As a student, I want to send a study request to a classmate of the same course, so that we can prepare the Mid-Term together.”

I	N	V	E	S	T
Independent	Negotiable	Valuable	Estimable	Small	Testable

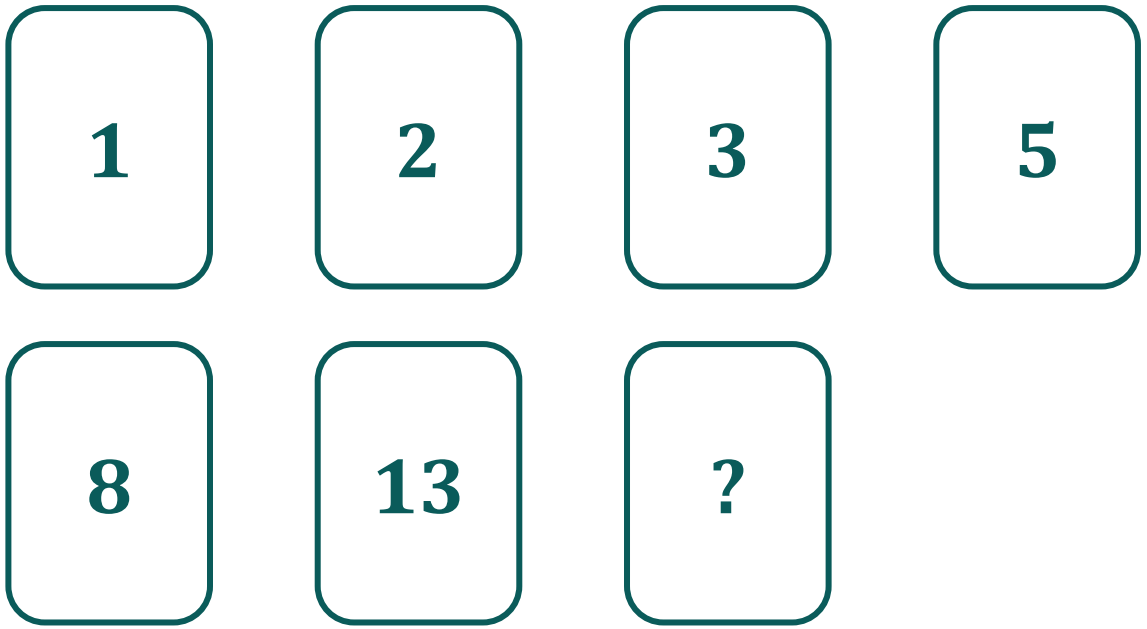
Acceptance criteria: Given, When, Then

- **Acceptance criteria**: conditions for accepting a story, written before the work
- Format from **BDD** (Behaviour-Driven Development), introduced by Dan North in 2006: **Given** a situation · **When** an action · **Then** an observable result
- Given I am logged in and Ana is in CS 330, when I send Ana a request for Thursday 14:00, then she sees it and I see “pending”
- Given Ana blocks requests, when I send one, then it is refused with a message
- Each criterion → a test in Block 7 · every requirement of Lab 05 traces back to a story

Given	I am logged in and Ana is enrolled in CS 330
When	I send Ana a study request for Thursday at 14:00
Then	Ana sees the request, and I see its status “pending”

Story points and planning poker

- **Story point**: relative size = effort + complexity + uncertainty, compared with the team's other stories — not hours
- People tend to compare two tasks more reliably than they predict one duration
- **Planning poker**: cards 0 · 1 · 2 · 3 · 5 · 8 · 13 · 20 · 40 · 100 · ?
- Choose in silence → reveal together → highest and lowest explain → vote again
- James Grenning (2002); Mike Cohn, *Agile Estimating and Planning* (2005)



Velocity — Numbers corner

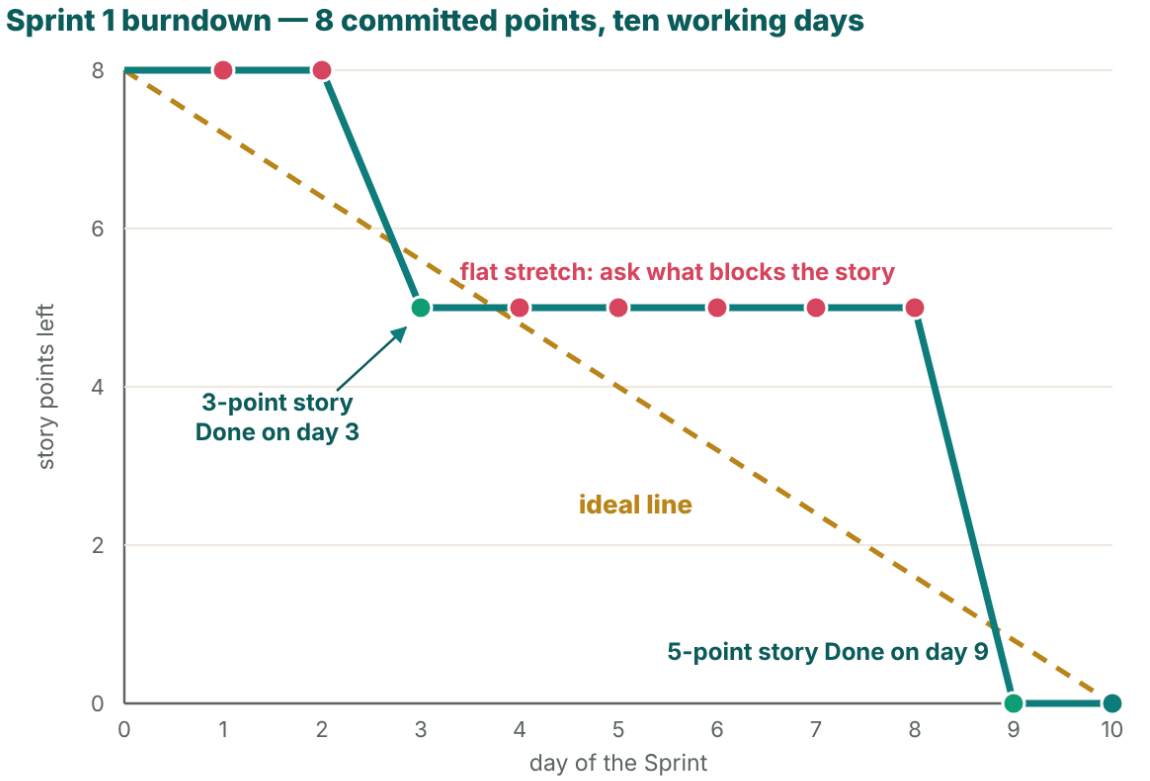
- **Velocity**: story points of the items that met the Definition of Done in one Sprint
- Measured, not promised: items started but not Done count zero
- Sprints needed = ⌈ remaining points ÷ velocity ⌉ (round up)
- 45 points ÷ 15 per Sprint = 3 Sprints · 18 ÷ 8 = 2.25 → 3 Sprints
- Velocity belongs to one team: do not compare teams
- A planning tool, not a target: a rewarded velocity inflates estimates

Sprints needed = ⌈ remaining story points ÷ velocity ⌉

45 ÷ 15 = 3 Sprints · 18 ÷ 8 = 2.25 → 3 Sprints

Reading a burndown chart

- **Burndown chart**: points remaining against the days of the Sprint, with an ideal straight line
- Above the ideal line = behind · below = ahead
- The actual line falls in steps: a story counts only when Done
- A long flat stretch: look for a blocked or an oversized story — at the Daily Scrum
- Burnup chart: work completed rising toward the total; shows changes of scope



Sprints needed = ⌈ remaining story points ÷ velocity ⌉ [ ] = round up

Before Sprint 1: ⌈18 ÷ 8⌉ = ⌈2.25⌉ = 3 Sprints (assumed velocity 8)

After Sprint 1: measured velocity 8, 10 points left → ⌈10 ÷ 8⌉ = 2 more Sprints

Velocity counts only the points of stories that are Done.

Lesson 4 after class: watch, try, run

▶ Watch — 15 minutes or shorter

★ How to write good User Stories in Agile 3:47 ▶ watch

The Agile Shop

What Are Story Points in Agile? (Agile Estimation Explained) 6:18 ▶ watch

Mountain Goat Software: Agile & Scrum Mastery

What is Planning Poker? Project Management in Under 5 7:23 ▶ watch

Online **PM** (Project Management) Courses - Mike Clayton

What are a Burndown Chart, a Burnup Chart, and Velocity? 7:19 ▶ watch

Online **PM** (Project Management) Courses - Mike Clayton

Real-life example

StudyBuddy: from eight stories to a forecast and a burndown

Comes back in:

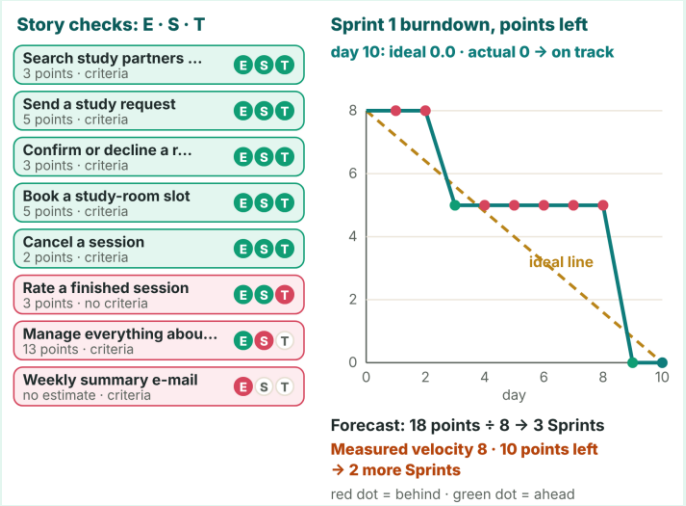
- Lab 05 / Block 5
- The starting point of Lab 05 next week
- Block 7
- Mid-Term Part B (velocity) and Part C (user stories with acceptance criteria)

Team Lynx and StudyBuddy are fictional.

Demo program

demo4_studybuddy_velocity_burndown.py

runs step by step in the module



5 flash-cards · ✎ 6 formative questions (4 multiple-choice, 2 multiple-select) — in the module and the Study Hub. **Formative: nothing is graded.**

Active break — Try-it: One INVEST (Independent, Negotiable, Valuable, Estimable, Small, Testable) user story from your Vision Card



4:00

0:44–0:48

1. Take one requirement from your Product Vision Card (Block 1).
2. Rewrite it as a user story: As a…, I want…, so that….
3. Add one Given/When/Then criterion, and check the story against the six letters of **INVEST** (Independent, Negotiable, Valuable, Estimable, Small, Testable).
4. On the count of three, estimate it with your fingers, planning-poker style: 1, 2, 3, 5 or 8.

5

LESSON 5 · 0:48–0:56 · 8 MINUTES · SLIDES 39–46

Process improvement

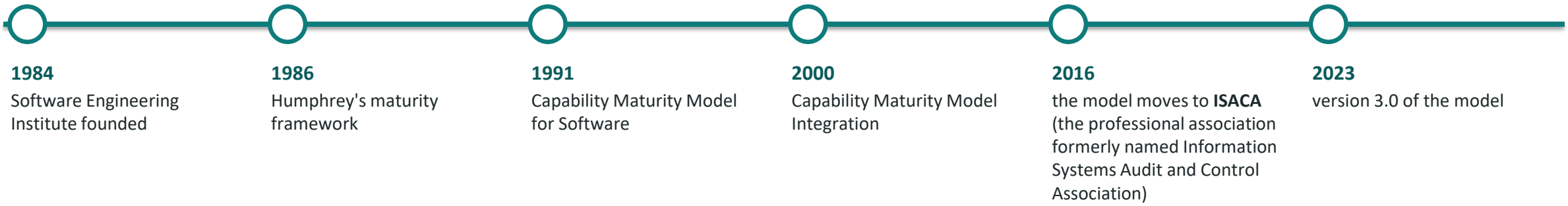
CMMI (Capability Maturity Model Integration) maturity levels, **ISO/IEC/IEEE 12207** (International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers standard 12207) and the **ISO/IEC 33000** (International Organization for Standardization / International Electrotechnical Commission standards series 33000) family, the improvement cycle — and the **LO1** (Learning Outcome 1) debate

Goal of the lesson

Define software process improvement; describe the improvement cycle (assess, plan, improve, measure) and the **IDEAL** (Initiating, Diagnosing, Establishing, Acting, Learning) model; name and describe the **CMMI** (Capability Maturity Model Integration) maturity levels; place **ISO/IEC/IEEE 12207** (International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers standard 12207) and the **ISO/IEC 33000** (International Organization for Standardization / International Electrotechnical Commission standards series 33000) family; argue for and against process improvement.

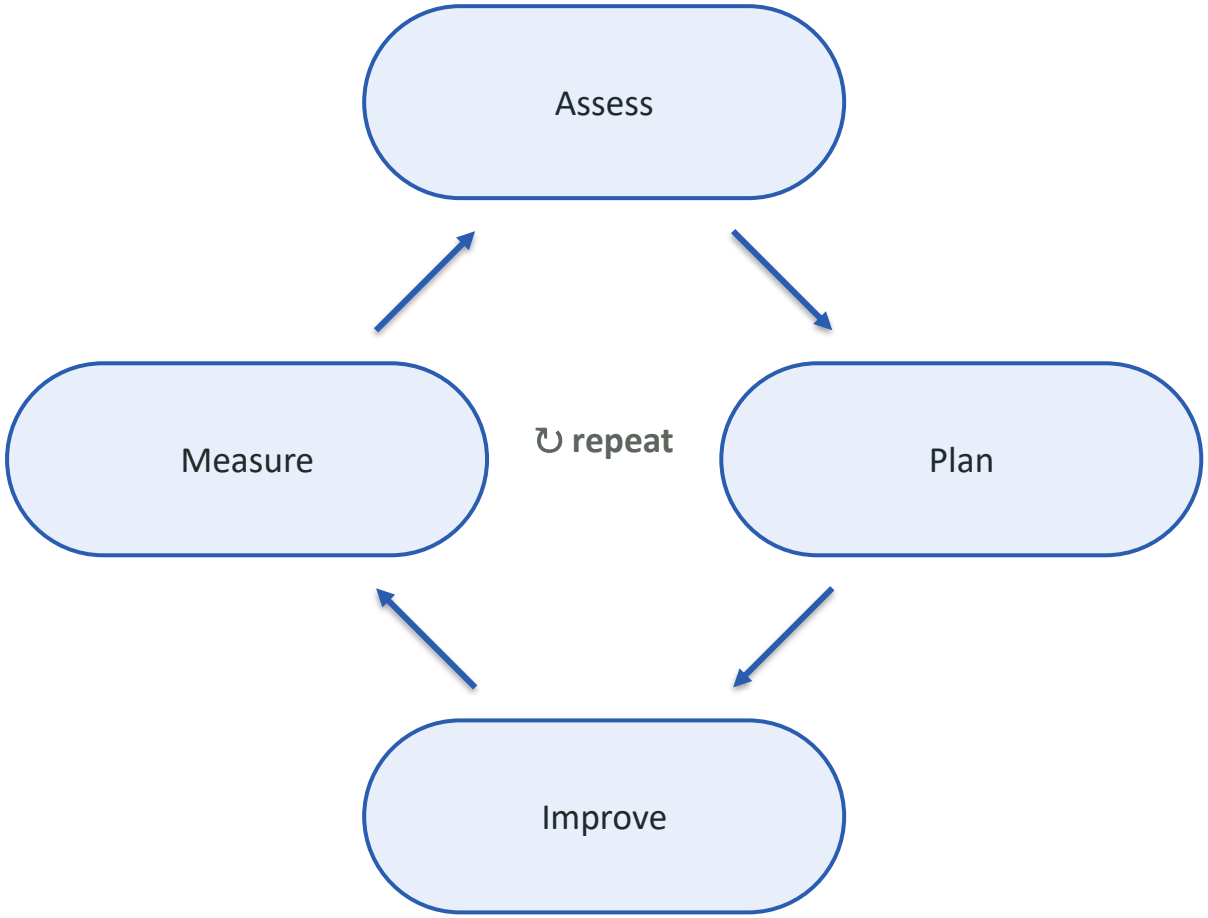
Software process improvement: what and why

- **SPI (Software Process Improvement)**: assess the process, find its weaknesses, change it, measure the effect
- Why: predictable schedules and quality; repeat what works; clients ask for evidence
- 1984: the **SEI** (Software Engineering Institute), a software research centre created by the United States Department of Defense
- 1986: Watts Humphrey's maturity framework, requested by the Air Force to evaluate software contractors
- 1991: **SW-CMM** (Capability Maturity Model for Software) · 2000: **CMMI** (Capability Maturity Model Integration) · since 2016 with **ISACA** (the professional association formerly named Information Systems Audit and Control Association) · today **CMMI** (Capability Maturity Model Integration) **V3.0** (version 3.0) (2023)



The improvement cycle

- **Assess** — compare the practices with a model; the difference is the gap
- **Plan** — a few changes for the most important part of the gap, with owners and dates
- **Improve** — introduce the changes, train, use them on real work
- **Measure** — evidence that the change helped
- The **SEI** (Software Engineering Institute) model of 1996: **IDEAL** (Initiating, Diagnosing, Establishing, Acting, Learning)
- The smallest improvement cycle: the Sprint Retrospective



CMMI (Capability Maturity Model Integration) maturity levels

Level	Name	What characterises it
0	Incomplete	work is improvised; it may or may not be finished
1	Initial	work gets done, but reactively; often late and over budget
2	Managed	projects are planned, performed, measured and controlled
3	Defined	standard processes for the organisation, tailored by each project
4	Quantitatively Managed	numeric goals for quality and performance, managed with data
5	Optimizing	continuous improvement based on data; quick adaptation

The international standards

- **ISO/IEC/IEEE 12207 (International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers standard 12207)** (2026 edition): the catalogue of software life-cycle processes, each with its purpose and outcomes
- It says what each process must achieve, not how a team must work; the 2026 edition replaced the 2017 one
- **ISO/IEC 33000 (International Organization for Standardization / International Electrotechnical Commission standards series 33000)** family: how to assess a process; **ISO/IEC 33020** (International Organization for Standardization / International Electrotechnical Commission standard 33020) (2019) defines capability levels 0–5
- Capability levels: incomplete · performed · managed · established · predictable · innovating
- It replaced **ISO/IEC 15504** (International Organization for Standardization / International Electrotechnical Commission standard 15504), known as **SPICE** (Software Process Improvement and Capability dEtermination)

12207

lists the processes

33000

measures how capable each process is

CMMI (Capability Maturity Model Integration)

groups practices into maturity levels for an organisation

The LO1 (Learning Outcome 1) debate: is process improvement worth it?

- For: predictable schedules and quality · repeat what works · new people learn the process · clients compare suppliers · measurements replace guesses
- Against: appraisals cost time and money · a process written for the appraisal, not for the work · heavy documentation slows small teams · a level says nothing direct about one product
- A common position: agile teams adopt **CMMI** (Capability Maturity Model Integration) practices without writing documents for their own sake
- In practice: a Definition of Done for quality assurance · a burndown chart for monitoring and control · a Git tag per Block for configuration management

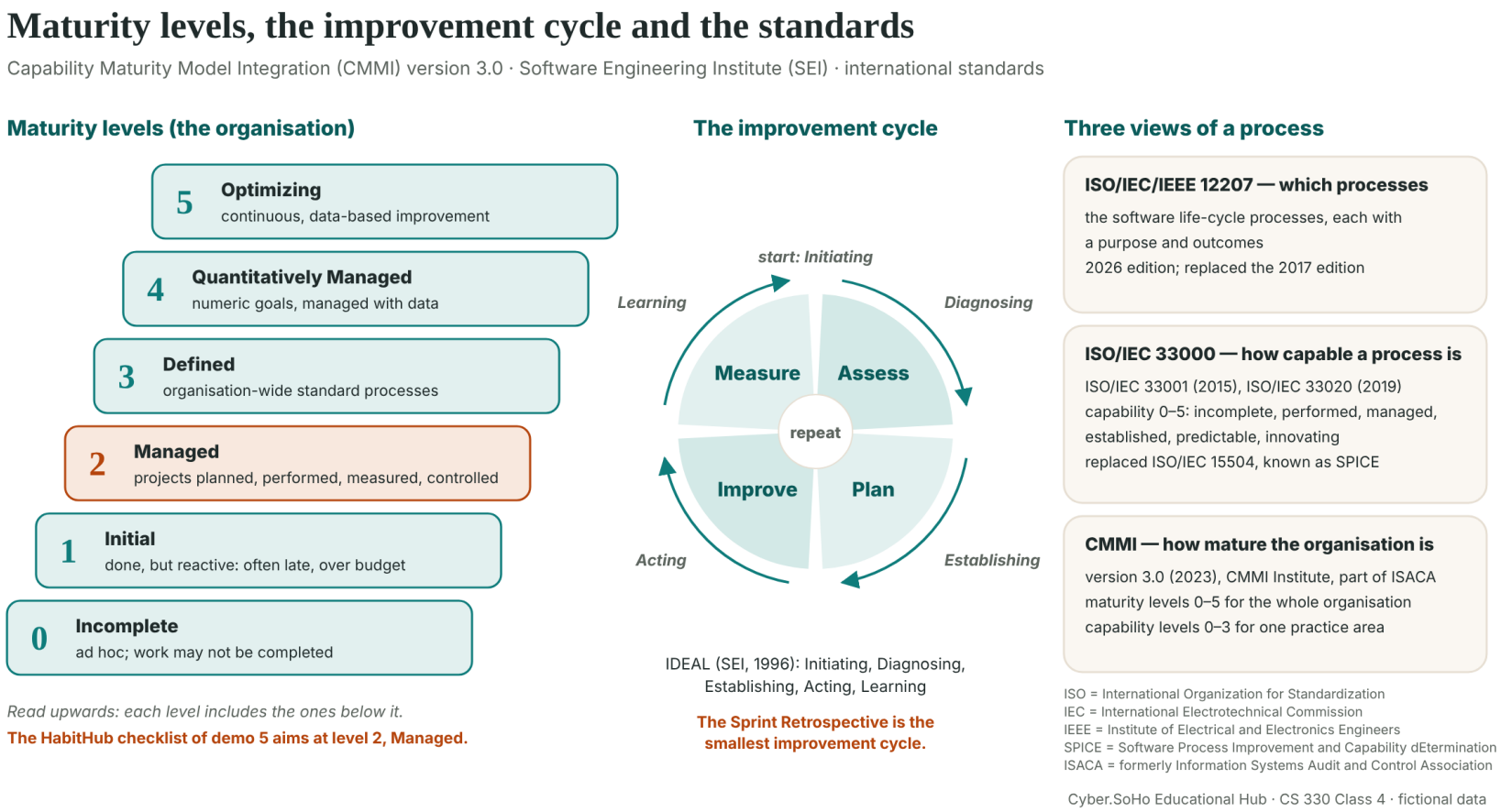
For

- predictable schedules and quality
- repeat what works
- clients can compare suppliers
- measurements, not guesses

Against

- appraisals cost time and money
- processes written but not used
- heavy for small teams
- a level is not a product guarantee

Figure 5 — Maturity levels, the improvement cycle and the standards



Left: the six maturity levels of **CMMI** (Capability Maturity Model Integration) **V3.0** (version 3.0). Centre: the improvement cycle and its **IDEAL** (Initiating, Diagnosing, Establishing, Acting, Learning) names. Right: how **ISO/IEC/IEEE 12207** (International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers standard 12207), the **ISO/IEC 33000** (International Organization for Standardization / International Electrotechnical Commission standards series 33000) family and **CMMI** (Capability Maturity Model Integration) divide the work.

Lesson 5 after class: watch, try, run

▶ Watch — 15 minutes or shorter

★ What is CMMI (Capability Maturity Model Integration) 5:43 ▶ watch

LearnEveryone

CMMI (Capability Maturity Model Integration) V3.0 (version 3.0) Practice Areas Explained 8:30 ▶ watch

Theoris

CMMI (Capability Maturity Model Integration) 2.0: From a Process Model to a Performance Model 9:10 ▶ watch

Welcome to **CMMI** (Capability Maturity Model Integration)-**TV** (television)!

Real-life example

HabitHub: two improvement cycles to a level-2-style checklist

Comes back in:

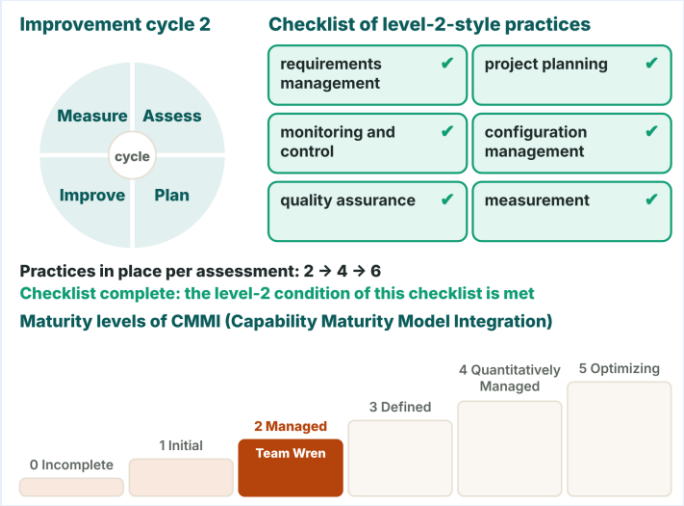
- The December finishing workshops: the written stand-up is monitoring and control; reviewing a teammate's file before the tag is quality assurance
- Lab 05
- Every Block: block3, block5, block7; Class 10
- The Final's LO1 (Learning Outcome 1) debate question

Team Wren and HabitHub are fictional.

Demo program

demo5_habithub_spi_cycle.py

runs step by step in the module



5 flash-cards · ✎ 6 formative questions (4 multiple-choice, 2 multiple-select) — in the module and the Study Hub. **Formative: nothing is graded.**

Active break — Zen-break, then flash-cards: Two quiet minutes, then the maturity levels



4:00

0:56–1:00

1. Two minutes of quiet: stand up, roll your shoulders, breathe in for four counts and out for six. Nothing to do — on purpose.
2. Then, in pairs, one minute each way: one partner says a number from 0 to 5; the other names the **CMMI** (Capability Maturity Model Integration) maturity level and gives one sentence about it.

Studio today: coached work time

- No new Block in Class 4
- 1:10–1:15 · feedback patterns from the Block 3 first versions
- 1:15–2:40 · team work on Block 3, the professor circulating
- 2:40–2:50 · optional, never graded: five user stories with acceptance criteria
- 2:50–3:00 · wrap-up and exit ticket

Block 3
final Sunday, Dec 20, 23:59 · 6.5 %

Optional practice
five user stories → Lab 05

Exit ticket
which practice first, and why?

Home Sprint and next week

- Finish Block 3 if needed
- Read Pressman and Maxim, chapters 3 and 28
- Optional: three velocity and burndown exercises in /docs/numbers.md
- Flash-cards: twelve new terms, last week's ten again
- Oct 20 · Class 5: requirements and Lab 05 — bring your stories

Read
chapters 3 and 28

Practise
velocity and burndown

Bring
five user stories

Links (1/4): course, legal and safety · agility and Scrum

Course, legal and safety

- Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy (version 2.0) ↗ cyber-soho.github.io
- VirusTotal — **URL** (Uniform Resource Locator, a web address) scanner ↗ virustotal.com
- VirusTotal — file scanner ↗ virustotal.com
- Google Transparency Report — Safe Browsing site status ↗ transparencyreport.google.com

Agility and Scrum

- Manifesto for Agile Software Development ↗ agilemanifesto.org
- Principles behind the Agile Manifesto ↗ agilemanifesto.org
- History: The Agile Manifesto ↗ agilemanifesto.org
- The 2020 Scrum Guide (Scrum Guides) ↗ scrumguides.org
- Scrum Guide Expansion Pack ↗ scrumexpansion.org
- Scrum.org ↗ scrum.org
- Agile Alliance — glossary ↗ agilealliance.org
- Government of Canada Digital Standards ↗ canada.ca
- Canadian Digital Service ↗ digital.canada.ca

Links (2/4): Extreme Programming, Kanban, planning and metrics

XP (Extreme Programming), Kanban, planning and metrics

- Agile Alliance — Pair Programming ↗ agilealliance.org
- Agile Alliance — Test-Driven Development ↗ agilealliance.org
- The Kanban Guide (May 2025) ↗ kanbanguides.org
- Kanban University — The Official Guide to The Kanban Method ↗ kanban.university
- Agile Alliance — Kanban Board ↗ agilealliance.org
- Little, J. D. C. (1961), A Proof for the Queuing Formula: L = λW ↗ pubsonline.informs.org
- Bill Wake — **INVEST** (Independent, Negotiable, Valuable, Estimable, Small, Testable) in Good Stories, and **SMART** (Specific, Measurable, Achievable, Relevant, Time-boxed) Tasks (2003) ↗ xp123.com
- Agile Alliance — **INVEST** (Independent, Negotiable, Valuable, Estimable, Small, Testable) ↗ agilealliance.org
- Mountain Goat Software — Planning Poker ↗ mountaingoatsoftware.com
- Mountain Goat Software — Story Points ↗ mountaingoatsoftware.com
- Agile Alliance — Velocity ↗ agilealliance.org
- Agile Alliance — Burndown Chart ↗ agilealliance.org
- Agile Alliance — Definition of Done ↗ agilealliance.org
- GitHub Docs — About Projects ↗ docs.github.com

Links (3/4): process improvement

Process improvement and standards

- The **CMMI** (Capability Maturity Model Integration) Institute ↗ cmmiinstitute.com
- The **CMMI** (Capability Maturity Model Integration) Institute — Levels of Capability and Performance ↗ cmmiinstitute.com
- **ISACA** (the professional association formerly named Information Systems Audit and Control Association) ↗ isaca.org
- Software Engineering Institute, Carnegie Mellon University ↗ sei.cmu.edu
- McFeeley (1996), **IDEAL** (Initiating, Diagnosing, Establishing, Acting, Learning): A User's Guide for Software Process Improvement ↗ sei.cmu.edu

Links (4/4): standards, textbook and research

Process improvement and standards (continued)

- **ISO/IEC/IEEE 12207** (International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers standard 12207), 2026 edition — Software life cycle processes ↗ iso.org
- **ISO/IEC/IEEE 12207** (International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers standard 12207), 2017 edition (withdrawn) ↗ iso.org
- **ISO/IEC 33001** (International Organization for Standardization / International Electrotechnical Commission standard 33001), 2015 edition — Process assessment, concepts and terminology ↗ iso.org
- **ISO/IEC 33020** (International Organization for Standardization / International Electrotechnical Commission standard 33020), 2019 edition — Process measurement framework for assessment of process capability ↗ iso.org
- International Electrotechnical Commission ↗ iec.ch
- Institute of Electrical and Electronics Engineers ↗ ieee.org

Textbook and research

- Pressman and Maxim, Software Engineering: A Practitioner's Approach, 9th edition (McGraw Hill) ↗ mheducation.com
- Boehm and Turner (2003), Using Risk to Balance Agile and Plan-Driven Methods ↗ doi.org

Legal notice, credits and sources

Legal notice

Use of this deck is governed by the **Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy** (version 2.0).

CS 330 is not a cybersecurity course: nothing here asks anyone to scan, test or touch a system they do not own. What applies: licence and copyright (section 4), third-party content (4.6), independence (5), running code on your own equipment (10), link and download safety (14, Annex C), fictional data only (6.7, 18.2), recording (17), academic integrity (18).

Check every link with **VirusTotal** before you open it.

The student acknowledgement form (Annex A, e-signature) is provided separately.

Credits

- Figures, animation, demo programs and examples: made for this lecture by Cyber.SoHo; all teams, products and numbers are fictional.
- The Scrum Guide (2020) is paraphrased, never copied; its authors publish it under the **CC BY-SA** (Creative Commons Attribution-ShareAlike licence) 4.0 licence.
- Videos and pages are linked, not copied; their names belong to their owners. No affiliation with any organisation named.

© 2026 Cyber.SoHo — From the Industry to the Classroom — Building the Next Generation of IT Pros.